

MUSTAFACODE

Lecture 18

Object Oriented Programming (CS304)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 18: Object Oriented Programming (CS304)

Object Oriented Programming (CS304)

Lecture No.18 — Assignment Operator, Self Assignment Problem, Friend Functions & Binary Operators

Table of Contents

1. Introduction
 2. Self Assignment Problem
 3. Why Self Assignment Causes Issues
 4. Solution of Self Assignment
 5. `this` Pointer Explanation
 6. Assignment Operator Return Type
 7. Constant Return Type
 8. `+=` Operator Overloading
 9. Friend Functions
 10. Non-Friend Operator Overloading
 11. Real-Life Examples
 12. Other Binary Operators
 13. Complete Practice Programs
 14. Important Interview Questions
 15. Summary
-

1. Introduction

Lecture 18 mein hum ne seekha:

- Self Assignment Problem
- Assignment Operator Overloading
- += Operator Overloading
- Friend Functions
- Non-Friend Operators
- Binary Operators

Ye concepts memory management aur operator overloading samajhne ke liye bohat important hain.

2. Self Assignment Problem

Self Assignment kya hota hai?

Jab ek object ko usi object ke equal assign kar diya jaye:

```
str1 = str1;
```

Isay Self Assignment kehte hain.

Example

```
#include <iostream>
#include <cstring>
using namespace std;

class String{

private:

    char* bufferPtr;
    int size;

public:

    String(const char* text = ""){

        size = strlen(text);

        bufferPtr = new char[size + 1];

        strcpy(bufferPtr, text);
    }

    void display(){

        cout << bufferPtr << endl;
    }

    String& operator=(const String& rhs){

        size = rhs.size;

        delete[] bufferPtr;

        bufferPtr = new char[size + 1];

        strcpy(bufferPtr, rhs.bufferPtr);

        return *this;
    }

    ~String(){

        delete[] bufferPtr;
    }
}
```

```
};  
  
int main(){  
  
    String str1("Fakhir");  
  
    str1 = str1;  
  
    str1.display();  
  
    return 0;  
}
```

Problem kya hogi?

Step-by-Step

Jab:

```
str1 = str1;
```

execute hota hai to:

Step 1

```
delete[] bufferPtr;
```

Memory delete ho jati hai.

Step 2

Program copy karne ki koshish karta hai:

```
strcpy(bufferPtr, rhs.bufferPtr);
```

Lekin:

- `rhs.bufferPtr`
- aur `bufferPtr`

dono same memory ko point kar rahe hote hain.

Aur woh memory already delete ho chuki hoti hai.

Result

Program:

- crash kar sakta hai
- garbage value de sakta hai
- memory access violation de sakta hai

Real-Life Example

Socho:

Aap ne notebook destroy kar di aur phir usi notebook se notes copy karne ki koshish ki.

Impossible.

Yahi Self Assignment Problem hai.

3. Solution of Self Assignment

Hum pehle check karenge ke dono objects same to nahi.

Correct Code

```
String& operator=(const String& rhs){  
  
    if(this != &rhs){  
  
        size = rhs.size;  
  
        delete[] bufferPtr;  
  
        bufferPtr = new char[size + 1];  
  
        strcpy(bufferPtr, rhs.bufferPtr);  
    }  
  
    return *this;  
}
```

Important Line

```
if(this != &rhs)
```

Iska Matlab

Expression	Meaning
this	Current object ka address
&rhs	Right side object ka address

Agar dono same hon:

```
this == &rhs
```

to assignment perform nahi hogi.

4. `this` Pointer Explanation

`this` kya hota hai?

`this` ek hidden pointer hota hai jo current object ko point karta hai.

Example

```
class Student{  
  
    public:  
  
        void show(){  
  
            cout << this;  
        }  
};
```

Agar:

```
Student s1;
```

to:

```
s1.show();
```

`this` ka matlab hoga:

```
&s1
```

5. Assignment Operator Return Type

Hum usually assignment operator ka return type:

```
String&
```


rakhte hain.

Why?

Taake chaining possible ho:

```
a = b = c;
```

Example

```
String s1, s2, s3;
```

```
s1 = s2 = s3;
```

Sab objects same value le lenge.

6. Constant Return Type

Agar hum return type:

```
const String&
```

kar dein:

```
const String& operator=(const String&);
```

to:

```
(s1 = s2) = s3;
```

allow nahi hoga.

Example

```
#include <iostream>
using namespace std;

class Test{

public:

    const Test& operator=(const Test&){

        return *this;
    }
};

int main(){

    Test t1, t2, t3;

    // Error
    (t1 = t2) = t3;
}
```

Primitive Types mein Allowed hai

```
int a, b, c;

(a = b) = c;
```

Ye valid hai.

7. Operator Overloading

Complex Class

```
class Complex{  
  
private:  
  
    double real;  
    double img;  
  
public:  
  
    Complex(double r = 0, double i = 0){  
  
        real = r;  
        img = i;  
    }  
  
    Complex& operator+=(const Complex& rhs);  
  
    void display();  
};
```

Function Definition

```
Complex& Complex::operator+=(const Complex& rhs){  
  
    real = real + rhs.real;  
  
    img = img + rhs.img;  
  
    return *this;  
}
```

Display Function

```
void Complex::display(){  
  
    cout << real << " + " << img << "i" << endl;  
}
```

Main Function

```
int main(){  
  
    Complex c1(2,3);  
  
    Complex c2(4,5);  
  
    c1 += c2;  
  
    c1.display();  
  
    return 0;  
}
```

Output

```
6 + 8i
```

Real-Life Example

```
wallet1 += wallet2;
```

Matlab:

wallet2 ke paise wallet1 mein add ho gaye.

8. Overloading `+=` with Double

```
Complex& operator+=(const double& rhs);
```

Definition

```
Complex& Complex::operator+=(const double& rhs){  
  
    real = real + rhs;  
  
    return *this;  
}
```

Example

```
Complex c1(5,2);  
  
c1 += 3.5;
```

Output

```
8.5 + 2i
```

Sirf real part update hua.

9. Friend Functions

Friend Function kya hoti hai?

Friend function class ke private members ko access kar sakti hai.

Example

```
class Test{  
  
private:  
  
    int num;  
  
public:  
  
    Test(){  
  
        num = 10;  
    }  
  
    friend void show(Test t);  
};  
  
void show(Test t){  
  
    cout << t.num;  
}
```

Problem with Friend Functions

Friend functions encapsulation ko weak kar deti hain.

Disadvantages

- Data vulnerability
 - Bugs
 - Tough debugging
 - Security issues
-

10. Non-Friend Operator Overloading

Hum operators ko bina friend banaye bhi overload kar sakte hain.

Example — `obj1 + obj2`

```
Complex operator+(const Complex& a, const Complex& b){  
  
    Complex t = a;  
  
    return t += b;  
}
```

Kaise kaam kar raha hai?

Step 1

Temporary object:

```
Complex t = a;
```

Step 2

```
t += b;
```

Step 3

Result return.

Example

```
Complex c1(2,3);  
  
Complex c2(4,5);  
  
Complex c3;  
  
c3 = c1 + c2;
```

Output

```
6 + 8i
```

11. `double + object`

```
Complex operator+(const double& a, const Complex& b){  
  
    Complex t = b;  
  
    return t += a;  
}
```

Example

```
3.5 + c1
```

12. `object + double`

```
Complex operator+(const Complex& a, const double& b){  
  
    Complex t = a;  
  
    return t += b;  
}
```

Example

```
c1 + 3.5
```

Real-Life Analogy

Example

```
wallet3 = wallet1 + wallet2;
```

Matlab:

Dono wallets ke paise mila kar ek naya wallet bana.

Original wallets same rahenge.

13. Other Binary Operators

Ye operators bhi isi tarah overload kiye ja sakte hain:

```
--=  
*=  
/=   
%=  
&=  
|=   
^=  
<<=  
>>=  
!=
```

Example of --=

```
Complex& operator--=(const Complex& rhs){  
  
    real = real - rhs.real;  
  
    img = img - rhs.img;  
  
    return *this;  
}
```

14. Complete Practice Program

```
#include <iostream>
using namespace std;

class Complex{

private:

    double real;
    double img;

public:

    Complex(double r = 0, double i = 0){

        real = r;
        img = i;
    }

    Complex& operator+=(const Complex& rhs){

        real += rhs.real;

        img += rhs.img;

        return *this;
    }

    Complex operator+(const Complex& rhs){

        Complex temp = *this;

        temp += rhs;

        return temp;
    }

    void display(){

        cout << real << " + " << img << "i" << endl;
    }
};

int main(){
```

```
Complex c1(2,3);

Complex c2(4,5);

Complex c3;

c3 = c1 + c2;

c3.display();

return 0;
}
```

Output

```
6 + 8i
```

15. Important Interview Questions

Q1: Self Assignment kya hota hai?

Jab object khud ko hi assign kare.

```
obj = obj;
```

Q2: Self Assignment dangerous kyun hota hai?

Kyuki memory delete hone ke baad wahi memory copy karne ki koshish hoti hai.

Q3: `this` pointer kya hota hai?

Current object ka hidden pointer.

Q4: Assignment operator reference return kyun karta hai?

Taake chaining possible ho.

Q5: Friend functions dangerous kyun hoti hain?

Kyuki private data directly access kar leti hain.

16. Summary

Lecture 18 Summary

Hum ne seekha:

- Self Assignment Problem
 - Self Assignment Solution
 - `this` Pointer
 - Assignment Operator Return Type
 - `+=` Operator Overloading
 - Friend Functions
 - Non-Friend Operators
 - Binary Operators
-

Final Tip

Operator overloading ka golden rule:

Code readable aur natural lagna chahiye.

Example:

```
c1 + c2
```

ye:

```
add(c1,c2)
```

se zyada readable hota hai.

MUSTAFACO